

Documentação — Tech do Bem — Sprint 4

Disciplina: Building Relational Database

Curso: 1TDSR — FIAP

Equipe:

RM	Nome
RM568542	Hugo Souza de Jesus
RM566815	Lucas Campanhã dos Santos
RM567010	Lucas Marcelino Pompeu

Sumário

1. **Objetivo e escopo do projeto**
 2. Modelagem — entidades e relacionamentos
 3. Estrutura DDL e constraints
 4. Carga de dados (DML — INSERTs)
 5. Atualização e remoção (UPDATE + DELETE)
 6. Relatórios (5 SELECTs da rubrica)
 7. Como executar
 8. Atendimento à rubrica

1. Objetivo e escopo do projeto

A **Tech do Bem** é uma ONG que oferta atendimento odontológico gratuito a populações em vulnerabilidade. O banco modelado neste repositório sustenta a plataforma de gestão da ONG, controlando:

- **Colaboradores** com 4 níveis hierárquicos (Administrador, Coordenador, Auxiliar, Estagiário).
 - **Dentistas** que prestam atendimento sob a responsabilidade de um colaborador.
 - **Pacientes** vinculados a um dentista responsável, com dados de geolocalização opcionais (CEP, bairro, lat/lng) para análise de demanda regional.
 - **Campanhas** organizadas pelos colaboradores.
 - **Atendimentos** ligando paciente, dentista e campanha, com **exames** opcionais por atendimento.
 - **Anotações polimórficas** que podem se referir a dentista, paciente ou atendimento.
 - **Notificações** trafegando entre colaboradores↔dentistas (interno) e dentistas↔pacientes (externo).

- **Solicitações** de acesso (internas e de cadastro externo, vindas pela tela de login).

O sistema completo é composto pelo backend Java Quarkus, frontend React+Vite, e uma API Python Flask com modelo preditivo de demanda — todos consumindo este banco Oracle.

2. Modelagem — entidades e relacionamentos

2.1 Lista de tabelas

Tabela	Propósito
T_COLABORADOR	Funcionários internos da ONG (Admin, Coord, Aux, Estag).
T_DENTISTA	Profissionais de odontologia, cada um sob responsabilidade de 1 colaborador.
T_PACIENTE	Pacientes atendidos, com endereço e coordenadas opcionais.
T_CAMPANHA	Eventos / campanhas organizadas pelos colaboradores.
T_ATENDIMENTO	Atendimentos efetivos: paciente × dentista × campanha + data + status.
T_EXAME	Exames vinculados a um atendimento.
T_NOTIFICACAO	Mensagens enviadas entre colaborador→dentista ou dentista→paciente.
T_SOLICITACAO	Pedidos de acesso (internos ou cadastro externo via tela de login).
T_ANOTACAO	Anotações polimórficas (autor: colaborador/dentista; sobre: dentista/paciente/atendimento).
T_CAMPANHA_ATENDIMENTO	Associativa para histórico de campanha vs atendimento.
T_EXAME_ATENDIMENTO	Associativa exame × atendimento.
T_ENVIA	Associativa para registrar envios de notificações.

2.2 Cardinalidades principais

- **T_COLABORADOR 1 — N T_DENTISTA** (cada dentista tem 1

colaborador responsável).

- **T_DENTISTA** 1 — N **T_PACIENTE** (cada paciente tem 1 dentista responsável).
- **T_COLABORADOR** 1 — N **T_CAMPANHA** (cada campanha é organizada por 1 colaborador).
- **T_ATENDIMENTO** N — 1 **T_PACIENTE**, N — 1 **T_DENTISTA**, N — 1 **T_CAMPANHA**.
- **T_EXAME** N — 1 **T_ATENDIMENTO**.
- **T_NOTIFICACAO** suporta dois fluxos via FKs anuláveis:
 - **col_to_den**: **id_colaborador** preenchido, **id_paciente** NULL.
 - **den_to_pac**: **id_paciente** preenchido, **id_colaborador** NULL.
- **T_SOLICITACAO** suporta duas variantes:
 - Interna: **id_solicitante** preenchido (FK para colaborador), campos ***_externo** NULL.
 - Externa: **id_solicitante** NULL, dados do candidato em **nome_externo**, **cpf_externo**, **email_externo**, **senha_externo**, **telefone_externo**.
- **T_ANOTACAO** polimórfica via **autor_tipo** ∈ {colaborador, dentista} e **sobre_tipo** ∈ {dentista, paciente, atendimento}.

3. Estrutura DDL e constraints

Toda a DDL está concentrada no script único e cobre os 5 tipos exigidos pela disciplina:

Constraint	Onde aparece
PK	Toda tabela possui ID_* como NUMBER PK (alimentado por sequence SEQ_*).
FK	T_DENTISTA.T_COLABORADOR_ID , T_COLABORADOR.ID_COLABORADOR , T_PACIENTE.ID_DENTISTA , T_ATENDIMENTO.T_PACIENTE_ID_PACIENTE + T_DENTISTA.ID_DENTISTA + ID_CAMPANHA , etc.
NN	Campos obrigatórios marcados NOT NULL (ex: NOME , CPF , EMAIL , SENHA , DATA_NASC).
UK	T_COLABORADOR.EMAIL , T_DENTISTA.CRO , T_DENTISTA.EMAIL , T_PACIENTE.CPF (quando não nulo).

CK	T_COLABORADOR.CARGO IN ('Administrador','Coordenador','Auxiliar','Estagiário'), T_PACIENTE.UF (regex 2 letras), T_DENTISTA.CRO (regex \d{6}-[A-Z]{2}), T_PACIENTE.CPF (regex 11 dígitos), T_ATENDIMENTO.STATUS IN (...), etc.
-----------	---

3.1 Idempotência (anti-conflito de execução)

O bloco inicial do script é uma anonymous PL/SQL block que **dropa todas as tabelas e sequences existentes em ordem reversa de dependência**, tolerando **ORA-00942** (tabela inexistente) e **ORA-02289** (sequence inexistente). Isso garante que o script possa ser executado **inúmeras vezes** sem precisar limpar o schema manualmente — atendendo ao requisito "criar a codificação DDL inicial para dropar as tabelas" da rubrica.

```
BEGIN
  FOR t IN (
    SELECT 'T_ANOTACAO' AS n FROM dual UNION ALL
    SELECT 'T_SOLICITACAO' FROM dual UNION ALL
    ...
  ) LOOP
    BEGIN
      EXECUTE IMMEDIATE 'DROP TABLE ' || t.n || '
CASCADE CONSTRAINTS';
      EXCEPTION WHEN OTHERS THEN
        IF SQLCODE != -942 THEN RAISE; END IF;
      END;
    END LOOP;
    ...
  END;
/
```

4. Carga de dados (DML — INSERTs)

O script popula o banco com volume realista pensado para uma demonstração com ML/análise regional:

Tabela	Linhas
T_COLABORADOR	11 (1 admin especial admin@admin.com + 10 colaboradores)

T_DENTISTA	24 (3 por colaborador-responsável, exceto estagiários)
T_PACIENTE	240 (10 por dentista)
T_CAMPANHA	12
T_ATENDIMENTO	80
T_EXAME	40
T_NOTIFICACAO	30
T_SOLICITACAO	12
T_ANOTACAO	15
T_CAMPANHA_ATENDIMENTO	10
T_EXAME_ATENDIMENTO	10
T_ENVIA	10

A rubrica exige no mínimo 7 linhas por tabela — entregamos volume bem superior para sustentar relatórios significativos.

Os pacientes têm coordenadas e bairros reais de São Paulo (sorteados do dataset [demanda_bairros.csv](#) usado no modelo de ML), permitindo análises geográficas no frontend.

5. Atualização e remoção (UPDATE + DELETE)

A rubrica exige 3 UPDATES e 3 DELETES — implementados após os INSERTs:

5.1 UPDATES

1. **Promoção de cargo** — Promove uma Auxiliar a Coordenadora.
2. **Alteração de status** — Marca um atendimento Agendado como Realizado.
3. **Atualização de contato** — Altera o telefone de um paciente.

5.2 DELETES

1. **Solicitação rejeitada antiga** — Remove pedidos de cadastro descartados.
2. **Notificação obsoleta** — Limpa uma notificação muito antiga já lida.
3. **Exame sem resultado** — Remove um exame que ficou sem laudo após cancelamento do atendimento.

6. Relatórios (5 SELECTs da rubrica)

Os 5 relatórios finais cobrem todos os tipos exigidos pela rubrica (60 pontos):

Relatório 1 — Classificação de dados (5 pts) — ORDER BY + GROUP BY

Top 10 dentistas com maior número de pacientes vinculados, ordenados em ordem decrescente. Demonstra agrupamento e ordenação.

Relatório 2 — Função numérica simples (5 pts)

Idade média, total, mais novo e mais velho dos pacientes — usando **AVG**, **COUNT**, **MIN**, **MAX** sobre **MONTHS_BETWEEN(SYSDATE, DATA_NASC) / 12**.

Relatório 3 — Função de grupo (15 pts) — **GROUP BY + HAVING**

Quantidade de atendimentos por status (Agendado / Realizado / Cancelado), com data do primeiro e último atendimento de cada status, filtrando grupos não vazios.

Relatório 4 — Sub-consulta (15 pts)

Dentistas cuja quantidade de pacientes é **maior que a média** geral de pacientes por dentista. Usa sub-consulta correlacionada (**SELECT COUNT(*) ... WHERE P.ID_DENTISTA = D.ID_DENTISTA**) e sub-consulta de agregação (**SELECT AVG(QTD) FROM (...)**).

Relatório 5 — Junção de tabelas (20 pts) — **INNER JOIN** × 3

Últimos 20 atendimentos detalhados unindo **T_ATENDIMENTO** × **T_PACIENTE** × **T_DENTISTA** × **T_CAMPANHA**, ordenados por data decrescente.

7. Como executar

1. Conecte-se à instância Oracle (testado no Oracle FIAP — oracle.fiap.com.br:1521/orcl).
 2. Abra o arquivo **Tech do Bem - SQL - sprint 4.sql** no SQL Developer, SQL*Plus, SQLcl ou DBeaver.
 3. Execute o script completo. O processo:
 - **Dropa** todas as tabelas e sequences (tolerando inexistência).
 - **Cria** o schema completo (12 tabelas + 9 sequences + constraints).
 - **Insere** ~750 registros distribuídos pelas tabelas.
 - **Executa** 3 UPDATES e 3 DELETES demonstrativos.
 - **Imprime** os 5 relatórios finais.
4. Confira os resultados no painel de saída do cliente Oracle.

O script é idempotente: pode rodar várias vezes sem precisar limpar o schema. Tempo médio de execução completa: ~10 segundos no Oracle FIAP.

8. Atendimento à rubrica

Critério Sprint 4 (Building Relational Database)	Pontos máximos	Status
Documentação da Sprint 3	10	 Atendido (este documento contém modelagem completa)

Estrutura DDL atualizada (PK, FK, NN, UK, CK)	10	✓ Atendido (12 tabelas, todas as constraints presentes)
INSERTs ≥ 7 por tabela	10	✓ Atendido (entregamos 10 a 240 por tabela)
3 UPDATEs + 3 DELETEs	10	✓ Atendido (linhas 822 – 857 do SQL)
Relatório com classificação de dados	5	✓ Atendido (Top 10 dentistas)
Relatório com função numérica simples	5	✓ Atendido (Estatísticas de idade)
Relatório com função de grupo	15	✓ Atendido (Atendimentos por status com HAVING)
Relatório com sub-consulta	15	✓ Atendido (Dentistas acima da média)
Relatório com junção de tabelas	20	✓ Atendido (3 INNER JOINS em um SELECT)
Total	100	Pontuação máxima esperada

Penalidades evitadas

Penalidade	Mitigação
Não usar Oracle SQL (-50%)	Todo o SQL é Oracle (NUMBER, VARCHAR2, SEQUENCE, MONTHS_BETWEEN, FETCH FIRST n ROWS ONLY, etc).
Não criar codificação DDL inicial para dropar tabelas (-50%)	Bloco PL/SQL no início faz DROP em ordem reversa de dependência, tolerando inexistência.
Dados que não correspondem com a coluna (-50%)	Todos os INSERTs respeitam os checks regex (CPF 11 dígitos, CRO \d{6}-[A-Z]{2}, UF de 2 letras).
Não identificar integrantes (-30%)	Cabeçalho do SQL contém os 3 RMs (em ordem alfabética por nome) + este documento.
Instrução com erro de execução (nota = 0)	Script testado em produção (Oracle FIAP) — 528 statements executados sem erro.

Relatório que não retorna dados (nota = 0)	Todos os 5 relatórios retornam linhas após a carga padrão.
---	---